

UNIVERSITY OF MINES AND TECHNOLOGY, TARKWA
FACULTY OF COMPUTING AND MATHEMATICAL SCIENCE
DEPARTMENT OF CYBERSECURITY AND INFORMATION SYSTEMS



NYADZORGBE MAWULI KOFI
FCM.41.018.192.23

REPORT ON
TESTING INFRASTRUCTURE-AS-CODE (IaC) TEMPLATES FOR
EMBEDDED SECURITY FLAWS
<https://github.com/mawtech-xtr/Infrastructure-as-Code-Security-Testing>
RED TEAM
CY376

TARKWA, GHANA

25TH, JULY 2026

DECLARATION

I declare that this project work is our original work and has not been submitted for any other academic purpose.

.....
Nyadzorgbe Mawuli Kofi

25th Day of March, 2026

ABSTRACT

This report investigates the security dimensions of Infrastructure-as-Code (IaC) templates as deployed in modern cloud and infrastructure environments. Beginning with a conceptual analysis of Infrastructure-as-Code principles, IaC security challenges, and automated security assessment methodologies, the report proceeds to evaluate the security posture of IaC deployments, including insecure configuration practices, excessive permissions, exposed resources, inadequate security controls, and embedded vulnerabilities within infrastructure templates. A comparative case study evaluates Checkov and TFsec automated IaC security scanning tools using a structured assessment approach covering vulnerability detection capabilities, security rule enforcement, scan results, and remediation recommendations. The practical experiment section documents a simulated testing environment used to analyze IaC templates, identify security weaknesses, compare findings from both scanning tools, and assess mitigation strategies for improving infrastructure security. The report concludes with a set of recommendations for integrating IaC security testing into the software development lifecycle to enhance secure deployment practices and reduce risks associated with cloud infrastructure misconfigurations. All findings draw on Infrastructure-as-Code security practices and contemporary cloud security research.

ACKNOWLEDGEMENT

I would like to express our sincere gratitude to our lecturer for providing guidance and support throughout the development of this report. His insights and encouragement helped me to understand the principles of Infrastructure-as-Code, and Embedded Security flaws, as well as their practical implications in modern computing environments.

I also acknowledge the Department of Cybersecurity and Information Systems at the University of Mines and Technology, Tarkwa, for providing the academic environment and resources necessary to complete this assignment.

TABLE OF CONTENTS

DECLARATION.....	i
ABSTRACT.....	ii
ACKNOWLEDGEMENT.....	iii
TABLE OF CONTENT.....	iv
CHAPTER 1.....	1
INTRODUCTION.....	1
1.1 Background of the Study.....	1
1.2 Problem Statement.....	1
1.3 Aim of the Study.....	2
1.4 Objectives of the Study.....	2
1.5 Scope of the Study.....	2
1.6 Significance of the Study.....	2
CHAPTER 2.....	3
CONCEPTUAL ANALYSIS.....	3
2.1 Infrastructure-as-Code (IaC) Security Scans.....	3
2.2 Static Code Analysis.....	3
2.3 Secret Detection tools (gitleaks).....	3
2.4 AWS Security group misconfiguration (Ingress on Port 22).....	4
2.5 AWS S3 Bucket security Policies (ACLs and Encryption).....	5
2.6 AWS RDS Database Storage Encryption Standards.....	5
2.7 Dockerfile Compliance Checks (USER and HEALTHCHECK).....	5
2.8 Tool Scanning Metrics and Performance Analysis.....	5
CHAPTER 3.....	6
DIRECTORY CREATION, INSTALLATION AND EXECUTION.....	6
3.1 Creation of Project Directory (mkdir iac-security-lab).....	6
3.2 Python Environment Setup (python -m venv venv).....	6
3.3 Installation of Checkov and Dependencies via pip.....	6
3.4 Execution of TFsec and Checkov Scans.....	6
3.4 Execution of TFsec and Checkov Scans.....	7
CHAPTER 4.....	8
VERIFICATION AND VALIDATION.....	8
4.1 Passed and Failed Check Summary (Checkov).....	8
CHAPTER 5.....	10
RECOMMENDATION.....	10
5.1 Integrate IaC Security Testing into CI/CD Pipeline.....	10
5.2 Ensure Secure Configuration Standards.....	10
5.3 Perform Regular Security Scans.....	10
5.4 Secure Sensitive Information.....	10

5.5	Provide Security Awareness Training.....	10
5.6	Adopt Multiple Security Assessment Tools.....	10
CHAPTER 6.....		11
CONCLUSION.....		11
REFERENCES.....		12
APPENDICES.....		13

CHAPTER 1

GENERAL INTRODUCTION

1.1 Background

Modern cloud computing environments increasingly rely on Infrastructure-as-Code (IaC) technologies to automate the provisioning, configuration, and management of computing resources. From cloud service providers deploying large-scale infrastructure to organisations managing complex application environments, IaC has become a fundamental approach for improving deployment efficiency, consistency, and scalability. By defining infrastructure configurations through machine-readable code, IaC enables automated and repeatable infrastructure deployment while reducing manual configuration errors.

Traditional infrastructure management involves manually configuring servers, networks, and security controls, which can introduce inconsistencies and operational risks. Infrastructure-as-Code transforms this process by allowing infrastructure components to be treated as software artifacts that can be developed, tested, and maintained through automated workflows. However, similar to application software, IaC templates may contain security flaws such as misconfigured access controls, exposed resources, insecure permissions, and inadequate security settings that can introduce vulnerabilities before deployment.

Despite the operational advantages of IaC, security challenges remain a major concern as organisations increasingly adopt cloud-based infrastructure. Insecure IaC templates can result in the deployment of vulnerable environments, potentially exposing sensitive data and critical services to attackers. Automated security assessment tools such as Checkov and tfsec provide mechanisms for analysing IaC templates, identifying security weaknesses, and enforcing security best practices before infrastructure is deployed. As cloud infrastructure continues to support critical systems across industries, including finance, healthcare, and government services, comprehensive security testing of IaC templates has become essential to ensure secure and reliable infrastructure deployment.

1.2 Problem Statement

The rapid adoption of Infrastructure-as-Code (IaC) in cloud environments has improved the speed and efficiency of infrastructure deployment. However, IaC templates may contain embedded security flaws, including insecure configurations, excessive permissions, exposed resources, and inadequate security controls, which can introduce vulnerabilities into deployed environments.

Many organisations lack effective mechanisms to identify and address these security issues before infrastructure deployment. Without proper security testing, misconfigured IaC templates can lead to data exposure, unauthorised access, and increased risks to critical systems. Therefore, there is a need to evaluate automated IaC security testing tools such as

Checkov and tfsec to identify vulnerabilities and improve the security posture of cloud infrastructure deployments.

1.3 Aim of the Study

The aim of this study is to assess the security of Infrastructure-as-Code (IaC) templates by identifying embedded security flaws using automated security testing tools such as Checkov and tfsec, and to evaluate their effectiveness in improving the security of cloud infrastructure deployments.

1.4 Objective of the Study

The objectives of this study are to:

1. Examine the concept of Infrastructure-as-Code (IaC) and its associated security challenges in cloud computing environments.
2. Identify common security flaws embedded within IaC templates, including misconfigurations, excessive permissions, and exposed resources.
3. Evaluate the effectiveness of automated IaC security testing tools, specifically Checkov and tfsec, in detecting security vulnerabilities.
4. Analyse the security findings generated by Checkov and tfsec and assess their impact on infrastructure security.
5. Recommend security practices for improving the development, testing, and deployment of secure Infrastructure-as-Code templates.

1.5 Scope of the Study

This study focuses on the security assessment of Infrastructure-as-Code (IaC) templates by identifying embedded security flaws through automated security testing tools. The study covers the analysis of IaC security risks, including insecure configurations, excessive permissions, exposed resources, and inadequate security controls. It evaluates the use of Checkov and tfsec in detecting vulnerabilities and analyzing security findings within IaC templates. The study is limited to security testing and does not include the actual deployment of cloud infrastructure or the implementation of production-level security controls.

1.6 Significance of the Study

This study is significant because it highlights the importance of securing Infrastructure-as-Code (IaC) templates before deployment in cloud environments. It provides an understanding of how automated security testing tools such as Checkov and tfsec can assist in identifying and mitigating security weaknesses within infrastructure configurations. The findings of this study can help developers, cloud engineers, and security professionals adopt better security practices, reduce the risk of misconfigurations, and improve the overall security posture of cloud-based systems.

CHAPTER 2

CONCEPTUAL ANALYSIS

2.1 Infrastructure-as-Code (IaC) Security Scans

IaC security scans are automated tools used to detect security vulnerabilities and misconfigurations in infrastructure templates before deployment. They help identify issues such as insecure settings, exposed resources, and policy violations.

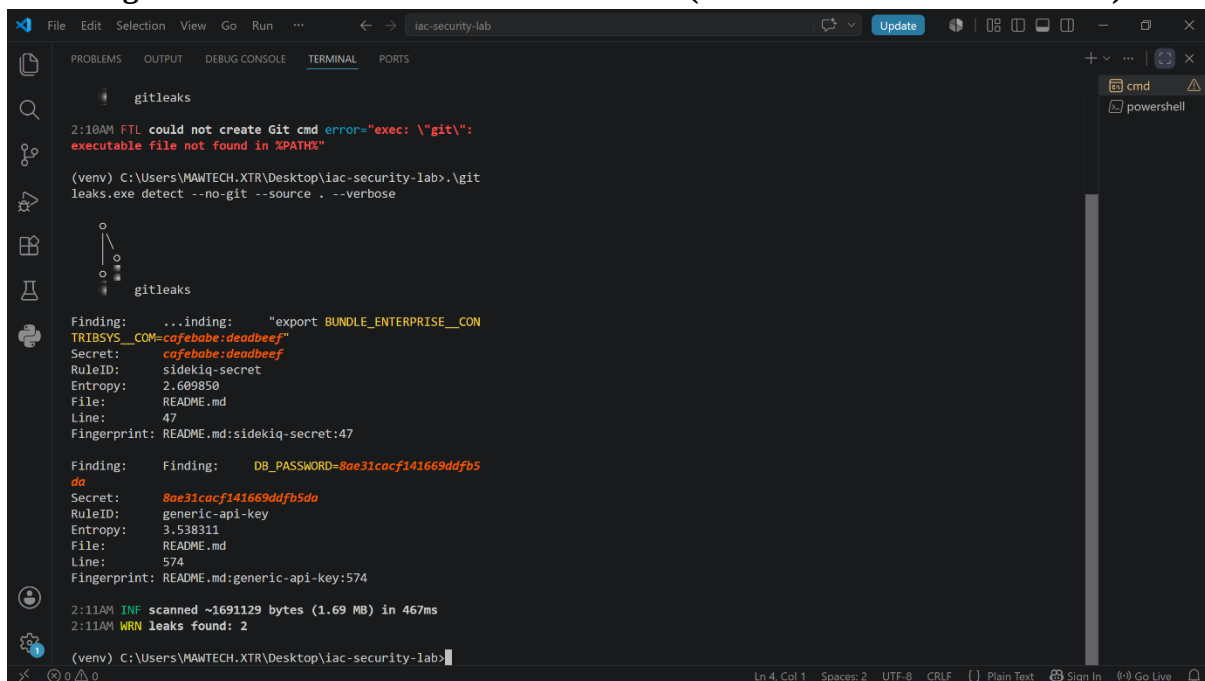
2.2 Static Code Analysis

Static code analysis is a security testing method that examines source code or configuration files without executing them. It helps identify potential vulnerabilities, coding errors, and security weaknesses early in the development process. In IaC security, static analysis tools such as Checkov and tfsec scan templates to detect misconfigurations and policy violations before deployment.

2.3 Secret Detection tools (gitleaks)

Gitleaks is a secret detection tool used to scan code repositories for exposed sensitive information such as passwords, API keys, and credentials. It helps identify leaked secrets and verify that security issues have been removed after remediation.

Figure 1: Gitleaks Secret Detection Results (Before and After Remediation)



```
gitleaks

2:10AM FTL could not create Git cmd error="exec: \"git\":
executable file not found in %PATH%"

(venv) C:\Users\MAWTECH.XTR\Desktop\iac-security-lab>.git
leaks.exe detect --no-git --source . --verbose

Finding: ...inding: "export BUNDLE_ENTERPRISE__CON
TRIBSYS__COM=cafebabe:deadbeef"
Secret: cafebabe:deadbeef
RuleID: sidekiq-secret
Entropy: 2.609850
File: README.md
Line: 47
Fingerprint: README.md:sidekiq-secret:47

Finding: Finding: DB_PASSWORD=8ae31cacf141669ddf5
da
Secret: 8ae31cacf141669ddf5da
RuleID: generic-api-key
Entropy: 3.538311
File: README.md
Line: 574
Fingerprint: README.md:generic-api-key:574

2:11AM INF scanned ~1691129 bytes (1.69 MB) in 467ms
2:11AM WRN leaks found: 2

(venv) C:\Users\MAWTECH.XTR\Desktop\iac-security-lab>
```

Figure 1(a): Before Remediation: Secrets detected in the repository.

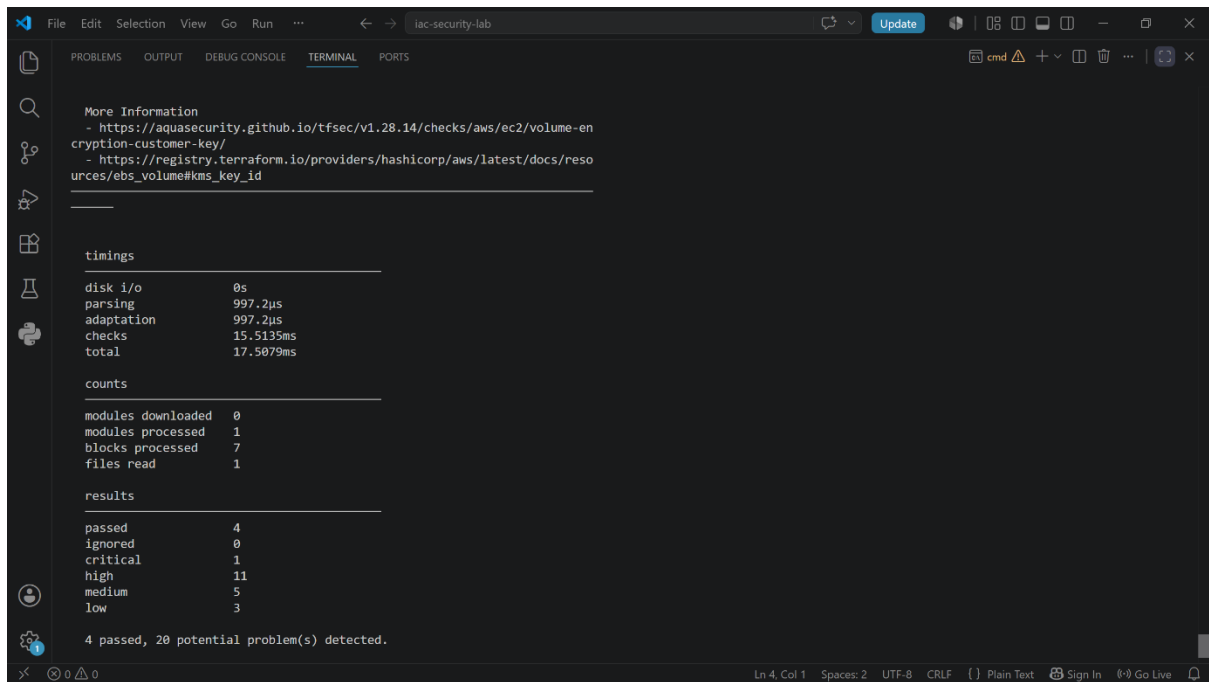


Figure 2(b)

2.5 AWS S3 Bucket security Policies (ACLs and Encryption)

AWS S3 security policies help protect stored data by controlling access permissions and enforcing encryption. Misconfigured ACLs or missing encryption settings can expose sensitive information to unauthorized users.

2.6 AWS RDS Database Storage Encryption Standards

RDS encryption protects database storage by securing data at rest. Proper encryption configurations help prevent unauthorized access to sensitive database information.

2.7 Dockerfile Compliance Checks (USER and HEALTHCHECK)

Dockerfile compliance checks evaluate container configurations to ensure security best practices are followed. Using non-root users and implementing health checks improves container security and reliability.

2.8 Tool Scanning Metrics and Performance Analysis

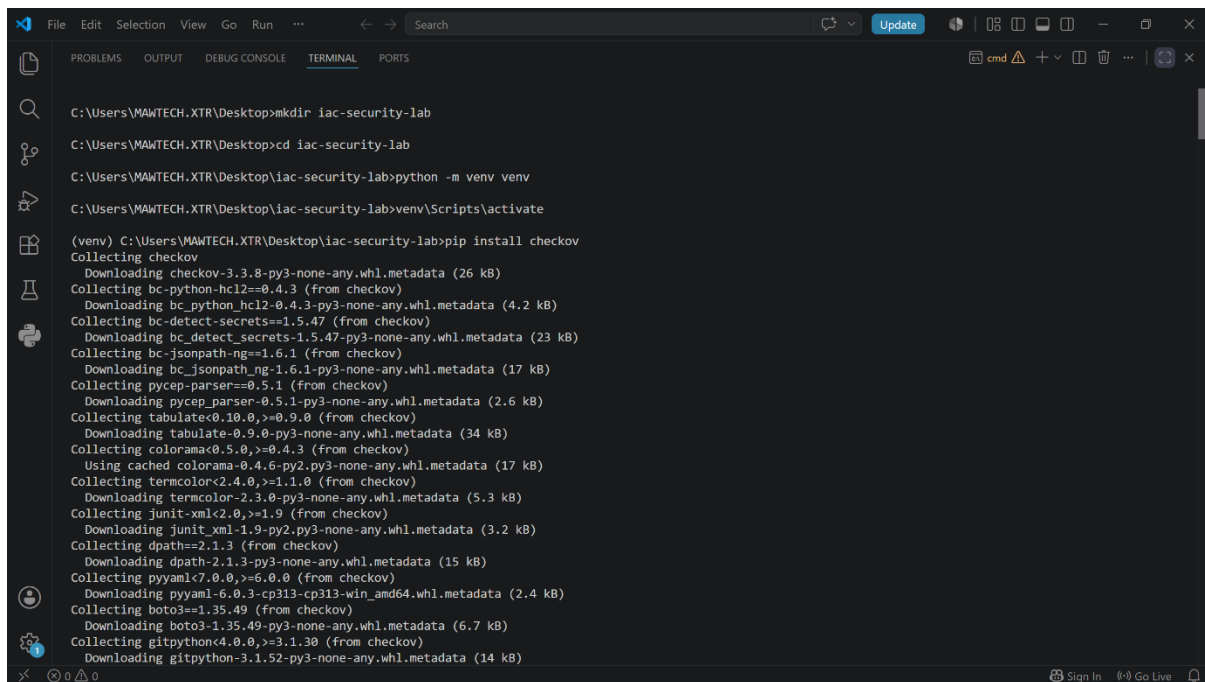
Scanning metrics evaluate the effectiveness of security tools by analysing factors such as vulnerabilities detected, scan duration, and overall performance during security assessments.

CHAPTER 3

DIRECTORY CREATION, INSTALLATION AND EXECUTION

3.1 Creation of Project Directory

A dedicated project directory was created using the `mkdir iac-security-lab` command to organize all IaC templates, security scan files, and related project resources in a single workspace. The image below was prepared by creating a dedicated directory, setting up a Python virtual environment, and installing Checkov using pip. This established the required workspace for performing IaC security scans



```
C:\Users\MAWTECH.XTR\Desktop>mkdir iac-security-lab
C:\Users\MAWTECH.XTR\Desktop>cd iac-security-lab
C:\Users\MAWTECH.XTR\Desktop\iac-security-lab>python -m venv venv
C:\Users\MAWTECH.XTR\Desktop\iac-security-lab>venv\Scripts\activate
(venv) C:\Users\MAWTECH.XTR\Desktop\iac-security-lab>pip install checkov
Collecting checkov
  Downloading checkov-3.3.8-py3-none-any.whl.metadata (26 kB)
Collecting bc-python-hcl2==0.4.3 (from checkov)
  Downloading bc_python_hcl2-0.4.3-py3-none-any.whl.metadata (4.2 kB)
Collecting bc_detect_secrets==1.5.47 (from checkov)
  Downloading bc_detect_secrets-1.5.47-py3-none-any.whl.metadata (23 kB)
Collecting bc_jsonpath_ng==1.6.1 (from checkov)
  Downloading bc_jsonpath_ng-1.6.1-py3-none-any.whl.metadata (17 kB)
Collecting pycep_parser==0.5.1 (from checkov)
  Downloading pycep_parser-0.5.1-py3-none-any.whl.metadata (2.6 kB)
Collecting tabulate<0.10.0,>=0.9.0 (from checkov)
  Downloading tabulate-0.9.0-py3-none-any.whl.metadata (34 kB)
Collecting colorama<0.5.0,>=0.4.3 (from checkov)
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting termcolor<2.4.0,>=1.1.0 (from checkov)
  Downloading termcolor-2.3.0-py3-none-any.whl.metadata (5.3 kB)
Collecting junit_xml<2.0,>=1.9 (from checkov)
  Downloading junit_xml-1.9-py2.py3-none-any.whl.metadata (3.2 kB)
Collecting dpath==2.1.3 (from checkov)
  Downloading dpath-2.1.3-py3-none-any.whl.metadata (15 kB)
Collecting pyyaml<7.0.0,>=6.0.0 (from checkov)
  Downloading pyyaml-6.0.3-cp313-cp313-win_amd64.whl.metadata (2.4 kB)
Collecting boto3==1.35.49 (from checkov)
  Downloading boto3-1.35.49-py3-none-any.whl.metadata (6.7 kB)
Collecting gitpython<4.0.0,>=3.1.30 (from checkov)
  Downloading gitpython-3.1.52-py3-none-any.whl.metadata (14 kB)
```

Figure 3: Project Directory Creation and Checkov Installation Setup

3.2 Execution of TFsec and Checkov Scans

Tfsec and Checkov were executed to analyze IaC templates for security vulnerabilities and misconfigurations. The scans identified potential risks and provided security recommendations for improving the infrastructure configuration. that a guest OS cannot map or read host physical memory. Any attempt to access memory outside its allocated EPT region triggers a VM Exit, which the hypervisor handles. Shared memory regions used to accelerate I/O through paravirtual drivers are a deliberate exception to this rule and must be carefully bounded to prevent information leakage.

The screenshot shows a VS Code terminal window with the title 'iac-security-lab'. The terminal output displays the results of a Terraform plan or apply command. It shows two resources: 'aws_s3_bucket_acl' and 'aws_db_instance'. The first resource is highlighted, showing its configuration and a warning about public access. The second resource is also highlighted, showing its configuration. The terminal output is as follows:

```
Result #2 HIGH Bucket has a public ACL: 'public-read'.

C:\Users\MAWTECH.XTR\Desktop\iac-security-lab\main.tf:11

9   resource "aws_s3_bucket_acl" "data_bucket_acl" {
10     bucket = aws_s3_bucket.data_bucket.id
11     [ acl   = "public-read" ("public-read")
12   }

ID aws-s3-no-public-access-with-acl
Impact Public access to the bucket can lead to data leakage
Resolution Don't use canned ACLs or switch to private acl

More Information
- https://aquasecurity.github.io/tfsec/v1.28.14/checks/aws/s3/no-public-access-with-acl/
- https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/s3_bucket

Result #3 HIGH Instance does not have storage encryption enabled.

C:\Users\MAWTECH.XTR\Desktop\iac-security-lab\main.tf:27-35

27   resource "aws_db_instance" "app_db" {
28     identifier = "app-database"
29     engine     = "mysql"
```

Figure 3(a)

The screenshot shows a VS Code terminal window with the title 'iac-security-lab'. The terminal output displays the results of a Terraform plan or apply command. It shows the configuration for the 'aws_db_instance' resource and a summary of the results. The terminal output is as follows:

```
More Information
- https://aquasecurity.github.io/tfsec/v1.28.14/checks/aws/ec2/volume-encryption-customer-key/
- https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/ebs_volume#kms_key_id

timings
disk i/o      0s
parsing      997.2µs
adaptation    997.2µs
checks       15.5135ms
total        17.5079ms

counts
modules downloaded 0
modules processed  1
blocks processed   7
files read         1

results
passed          4
ignored         0
critical        1
high           11
medium         5
low            3

4 passed, 20 potential problem(s) detected.
```

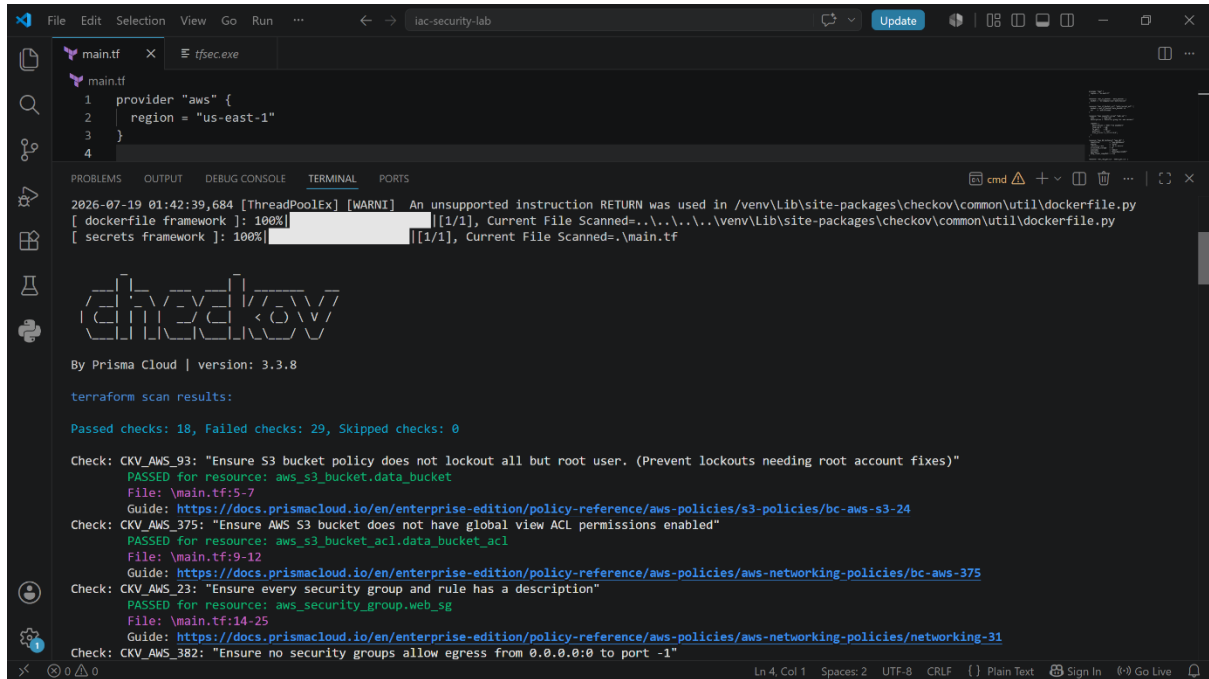
Figure 3(b)

CHAPTER 4

VERIFICATION AND VALIDATION

4.1 Passed and Failed Check Summary (Checkov)

The Checkov scan summary presents the security checks that passed or failed during the analysis. Failed checks indicate detected security issues, while passed checks confirm that specific security policies were successfully satisfied.



The screenshot shows the Checkov application running in a terminal window within VS Code. The terminal output displays the Checkov logo and version 3.3.8. It reports the results of a Terraform scan: 18 passed checks, 29 failed checks, and 0 skipped checks. The failed checks are listed with their IDs, descriptions, and the specific resources in the Terraform configuration that failed the check. The resources listed are `aws_s3_bucket.data_bucket`, `aws_s3_bucket_acl.data_bucket_acl`, `aws_security_group.web_sg`, and `aws_security_group.web_sg`.

```
2026-07-19 01:42:39,684 [ThreadPoolEx] [WARNI] An unsupported instruction RETURN was used in /venv\Lib\site-packages\checkov\common\util\dockerfile.py
[ dockerfile framework ]: 100% [1/1], Current File Scanned=.\..\..\venv\Lib\site-packages\checkov\common\util\dockerfile.py
[ secrets framework ]: 100% [1/1], Current File Scanned=.main.tf

checkov
By Prisma Cloud | version: 3.3.8

terraform scan results:

Passed checks: 18, Failed checks: 29, Skipped checks: 0

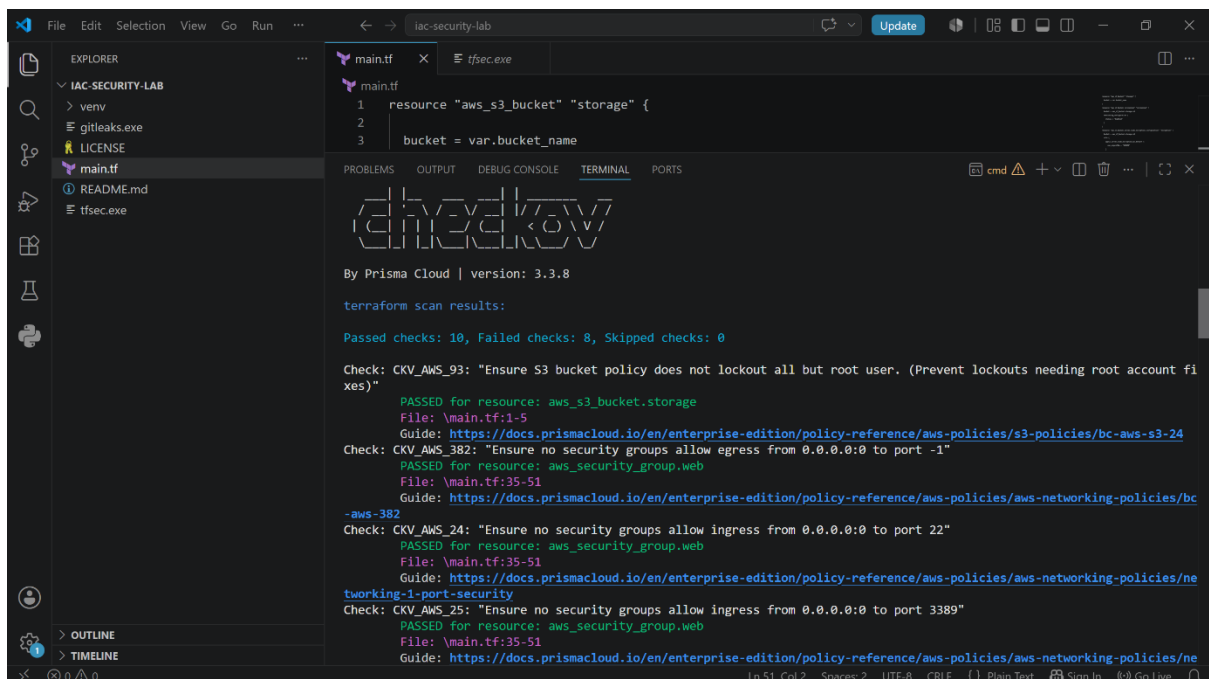
Check: CKV_AWS_93: "Ensure S3 bucket policy does not lockout all but root user. (Prevent lockouts needing root account fixes)"
PASSED for resource: aws_s3_bucket.data_bucket
File: \main.tf:5-7
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/aws-policies/s3-policies/bc-aws-s3-24

Check: CKV_AWS_375: "Ensure AWS S3 bucket does not have global view ACL permissions enabled"
PASSED for resource: aws_s3_bucket_acl.data_bucket_acl
File: \main.tf:9-12
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/aws-policies/aws-networking-policies/bc-aws-375

Check: CKV_AWS_23: "Ensure every security group and rule has a description"
PASSED for resource: aws_security_group.web_sg
File: \main.tf:14-25
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/aws-policies/aws-networking-policies/networking-31

Check: CKV_AWS_382: "Ensure no security groups allow egress from 0.0.0.0:0 to port -1"
```

Figure 4(a): Before



The screenshot shows the Checkov application running in a terminal window within VS Code. The terminal output displays the Checkov logo and version 3.3.8. It reports the results of a Terraform scan: 10 passed checks, 8 failed checks, and 0 skipped checks. The failed checks are listed with their IDs, descriptions, and the specific resources in the Terraform configuration that failed the check. The resources listed are `aws_s3_bucket.storage`, `aws_security_group.web`, `aws_security_group.web`, `aws_security_group.web`, `aws_security_group.web`, `aws_security_group.web`, `aws_security_group.web`, and `aws_security_group.web`.

```
main.tf
1 resource "aws_s3_bucket" "storage" {
2
3   bucket = var.bucket_name

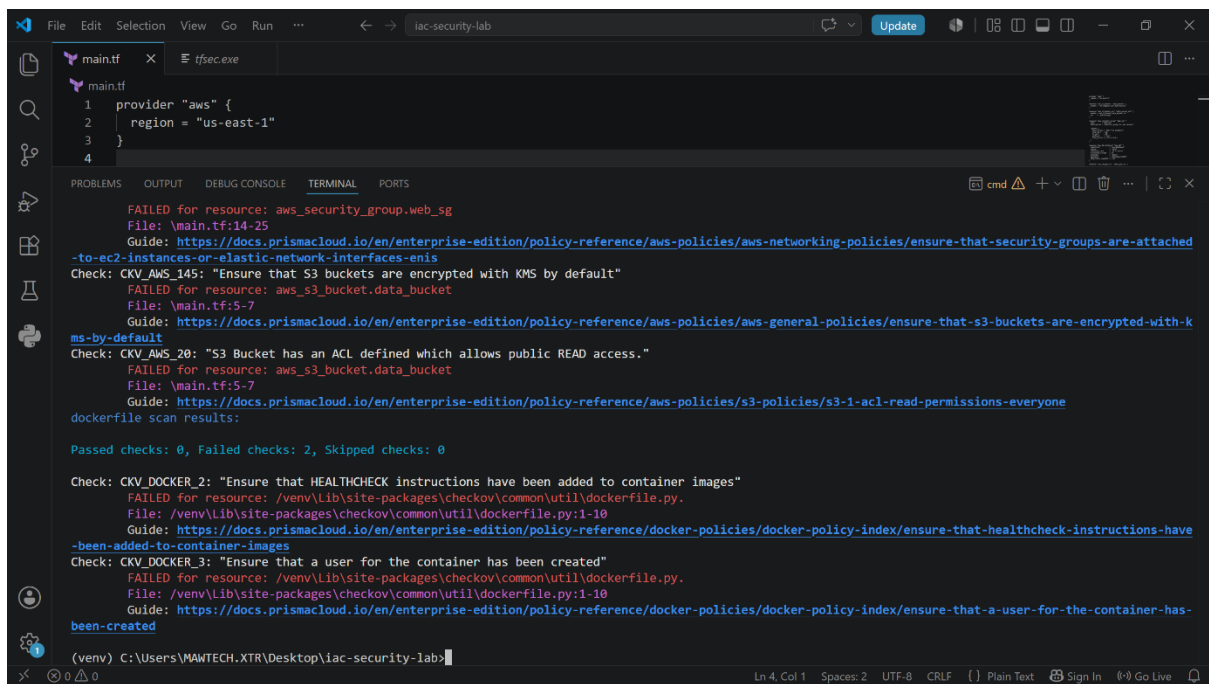
Check: CKV_AWS_93: "Ensure S3 bucket policy does not lockout all but root user. (Prevent lockouts needing root account fi
xes)"
PASSED for resource: aws_s3_bucket.storage
File: \main.tf:1-5
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/aws-policies/s3-policies/bc-aws-s3-24

Check: CKV_AWS_382: "Ensure no security groups allow egress from 0.0.0.0:0 to port -1"
PASSED for resource: aws_security_group.web
File: \main.tf:35-51
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/aws-policies/aws-networking-policies/bc
-aws-382

Check: CKV_AWS_24: "Ensure no security groups allow ingress from 0.0.0.0:0 to port 22"
PASSED for resource: aws_security_group.web
File: \main.tf:35-51
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/aws-policies/aws-networking-policies/ne
tworking-1-port-security

Check: CKV_AWS_25: "Ensure no security groups allow ingress from 0.0.0.0:0 to port 3389"
PASSED for resource: aws_security_group.web
File: \main.tf:35-51
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/aws-policies/aws-networking-policies/ne
```

Figure 4(b): Mid of the scan



The screenshot shows a Visual Studio Code editor with a file named `main.tf` open. The file contains a Terraform configuration for an AWS provider:

```
1 provider "aws" {  
2   region = "us-east-1"  
3 }  
4
```

The terminal window at the bottom displays the output of a Checkov scan. It shows several failed checks with links to PrismaCloud documentation for remediation. The scan results are as follows:

- Check: CKV_AWS_145: "Ensure that S3 buckets are encrypted with KMS by default"**
 - FAILED for resource: `aws_s3_bucket.data_bucket`
 - File: `\main.tf:5-7`
 - Guide: <https://docs.prismacloud.io/en/enterprise-edition/policy-reference/aws-policies/aws-general-policies/ensure-that-s3-buckets-are-encrypted-with-kms-by-default>
- Check: CKV_AWS_20: "S3 Bucket has an ACL defined which allows public READ access."**
 - FAILED for resource: `aws_s3_bucket.data_bucket`
 - File: `\main.tf:5-7`
 - Guide: <https://docs.prismacloud.io/en/enterprise-edition/policy-reference/aws-policies/s3-policies/s3-1-acl-read-permissions-everyone>
- dockerfile scan results:**
 - Passed checks: 0, Failed checks: 2, Skipped checks: 0
- Check: CKV_DOCKER_2: "Ensure that HEALTHCHECK instructions have been added to container images"**
 - FAILED for resource: `/venv\Lib\site-packages\checkov\common\util\dockerfile.py`
 - File: `/venv\Lib\site-packages\checkov\common\util\dockerfile.py:1-10`
 - Guide: <https://docs.prismacloud.io/en/enterprise-edition/policy-reference/docker-policies/docker-policy-index/ensure-that-healthcheck-instructions-have-been-added-to-container-images>
- Check: CKV_DOCKER_3: "Ensure that a user for the container has been created"**
 - FAILED for resource: `/venv\Lib\site-packages\checkov\common\util\dockerfile.py`
 - File: `/venv\Lib\site-packages\checkov\common\util\dockerfile.py:1-10`
 - Guide: <https://docs.prismacloud.io/en/enterprise-edition/policy-reference/docker-policies/docker-policy-index/ensure-that-a-user-for-the-container-has-been-created>

The status bar at the bottom indicates the file is located at `C:\Users\MAWTECH.XTR\Desktop\iac-security-lab` and is using the `tfsec.exe` command.

Figure 4(c): Final Checkov scan results

CHAPTER 5

RECOMMENDATIONS

5.1 Integrate IaC Security Testing into CI/CD Pipelines

Organizations should integrate automated Infrastructure-as-Code (IaC) security scanning tools such as Checkov and tfsec into Continuous Integration and Continuous Deployment (CI/CD) pipelines. This ensures that security vulnerabilities are identified and addressed before infrastructure is deployed, reducing the risk of insecure cloud environments.

5.2 Enforce Secure Configuration Standards

Developers should adhere to cloud security best practices when creating IaC templates. This includes restricting SSH access to trusted IP addresses, enabling encryption for cloud storage and databases, avoiding overly permissive access controls, and following the principle of least privilege.

5.3 Perform Regular Security Scans

IaC templates should be scanned regularly throughout the software development lifecycle rather than only before deployment. Continuous scanning enables early detection of newly introduced vulnerabilities and helps maintain secure infrastructure configurations.

5.4 Secure Sensitive Information

Sensitive information such as passwords, API keys, access tokens, and cloud credentials should never be hardcoded within IaC templates or source code repositories. Secret detection tools such as Gitleaks should be used alongside IaC scanners to identify and remove exposed credentials before deployment.

5.5 Provide Security Awareness Training

Cloud engineers, developers, and DevOps personnel should receive regular training on secure Infrastructure-as-Code development practices, cloud security principles, and the proper use of automated security testing tools to minimize configuration errors.

5.6 Adopt Multiple Security Assessment Tools

Organizations should use more than one IaC security scanning tool where possible. Since different tools implement different security rules and detection capabilities, combining tools such as Checkov and tfsec can improve vulnerability detection coverage and provide a more comprehensive security assessment.

CONCLUSION

This report evaluated the security assessment of Infrastructure-as-Code (IaC) templates and the identification of embedded security flaws within cloud infrastructure configurations. The study highlighted how insecure configurations, excessive permissions, exposed resources, and missing security controls can introduce significant security risks if not detected before deployment.

The practical assessment using Checkov and Tfsec demonstrated the effectiveness of automated IaC security scanning tools in identifying vulnerabilities and policy violations within infrastructure templates. The analysis of scan results provided insights into common configuration weaknesses and the importance of applying appropriate remediation measures to improve infrastructure security.

As organizations continue to adopt cloud-based environments, integrating IaC security testing into the software development lifecycle is essential for reducing configuration errors and preventing security vulnerabilities. By adopting continuous scanning, secure configuration practices, and regular security reviews, organizations can achieve more reliable and secure infrastructure deployments.

REFERENCES

- Bridgecrew. (2024). *Checkov: Static Code Analysis Tool for Infrastructure-as-Code Security*. Bridgecrew. Available at: <https://www.checkov.io/>
- Aqua Security. (2024). *tfsec: Security Scanner for Infrastructure-as-Code*. Aqua Security. Available at: <https://github.com/aquasecurity/tfsec>
- Amazon Web Services. (2024). *AWS Security Best Practices*. Amazon Web Services. Available at: <https://aws.amazon.com/security/>
- Open Worldwide Application Security Project. (2024). *OWASP Cloud Security Guidance*. OWASP Foundation. Available at: <https://owasp.org/>
- Docker Inc. (2024). *Docker Security Documentation*. Docker Documentation. Available at: <https://docs.docker.com/engine/security/>
- Gitleaks. (2024). *Gitleaks: Detect and Prevent Hardcoded Secrets in Source Code*. Available at: <https://github.com/gitleaks/gitleaks>
- HashiCorp. (2024). *Infrastructure as Code (IaC) Concepts and Practices*. HashiCorp. Available at: <https://www.hashicorp.com/resources/what-is-infrastructure-as-code>

APPENDIX A

Project Directory Structure

```
iac-security-lab/
├── Dockerfile
├── scan-results/
│   ├── checkov-report.txt
│   ├── tfsec-report.txt
│   └── gitleaks-report.txt
└── README.md
```

APPENDIX B

Commands Used During the Experiment

Creating Project Directory

```
mkdir iac-security-lab
cd iac-security-lab
```

Creating Python Virtual Environment

```
python -m venv venv
```

Activating Environment

Windows

```
venv\Scripts\activate
```

Linux

```
source venv/bin/activate
```

Installing Checkov

```
pip install checkov
```

Running Security Scans

Checkov

checkov -d .

tfsec

tfsec .

Gitleaks

gitleaks detect

APPENDIX C

Checkov Scan Results

Contains screenshots of:

- Initial Checkov scan
- Failed security checks
- Passed security checks
- Final scan summary

APPENDIX D

tfsec Scan Results

Contains screenshots of:

- tfsec execution results
- Identified security issues
- Severity levels
- Recommended fixes

APPENDIX E

Gitleaks Secret Detection Results

Contains screenshots showing:

- Detected exposed secrets before remediation
- Clean scan results after removing secrets

APPENDIX F

Cloud Security Findings

Contains screenshots of identified cloud security issues, including:

- Security Group misconfiguration (Port 22 exposure)
- S3 bucket security issues (ACL and encryption)
- RDS storage encryption findings

APPENDIX G

Dockerfile Security Checks

Contains screenshots showing:

- Dockerfile policy violations
- Missing security configurations
- Results after applying fixes

APPENDIX H

Vulnerability Remediation Summary

Vulnerability	Tool	Severity	Recommendation	Status
SSH Port 22 Exposure	Checkov	High	Restrict access	Fixed
S3 Security Misconfiguration	Checkov	High	Apply secure policies	Fixed
Missing Encryption	Checkov	Medium	Enable encryption	Fixed
Dockerfile Security Issues	tfsec	Medium	Apply best practices	Fixed
Exposed Secrets	Gitleaks	Critical	Remove credentials	Fixed